# ACTIVITY 1: DATASET PREPARATION

Unit I – Foundations & Data Preparation | Student Performance Dataset (Mathematics)

## 1. Introduction

Data preparation is a fundamental stage of data science. Raw datasets must be inspected, cleaned, transformed, and organized before meaningful analysis can be performed. A dataset may contain missing values, duplicate records, inconsistent text, unsuitable data types, and numerical features measured on different scales. If these issues are not addressed, they can affect the quality and interpretation of analysis.

This activity uses the Student Performance Mathematics dataset to demonstrate loading, inspection, statistical description, data-quality checks, exploratory data analysis (EDA), categorical encoding, numerical scaling, and exporting a prepared CSV file. The activity focuses on data preparation; no prediction model is developed.

## 2. Objectives

• Select and load a suitable tabular dataset.

• Inspect the structure, columns, data types, and descriptive statistics.

• Check for missing values and duplicate records.

• Perform basic cleaning and consistency checks.

• Explore the data using statistical visualizations.

• Encode categorical features into numerical form.

• Scale numerical input features.

• Save the prepared dataset in CSV format.

## 3. Dataset Description

**Dataset:** Student Performance – Mathematics
**Source:** UCI Machine Learning Repository
**Dataset page:** https://archive.ics.uci.edu/dataset/320/student+performance
**Expected mathematics subset size:** 395 records and 33 attributes.

The dataset contains information about students' personal and family backgrounds, school support, study habits, social activities, and academic results. The Mathematics file is one subject-specific subset of the broader Student Performance dataset.

| Feature | Description |
|---|---|
| school | School attended by the student |
| sex | Student's recorded sex category |
| age | Student's age |
| address | Urban or rural residence category |
| famsize | Family-size category |
| Medu / Fedu | Mother's / father's education level |
| studytime | Weekly study-time category |
| failures | Number of past class failures |
| schoolsup | Whether extra school support is provided |
| **Feature** | **Description** |
| internet | Whether internet access is available at home |
| G1 / G2 / G3 | First-period, second-period, and final grades |

The complete dataset contains additional attributes relating to family, school, lifestyle, and student support. The target-like final grade field is G3.

## 4. Tools and Technologies

| Tool / Library | Purpose |
|---|---|
| Python | Programming language used for data preparation |
| Google Colab / Jupyter | Environment for writing and running notebook cells |
| Pandas | Loading, inspecting, cleaning, and exporting tabular data |
| NumPy | Numerical operations and data-type selection |
| Matplotlib | Creating plots |
| Seaborn | Creating statistical visualizations |
| Scikit-learn | Standardizing numerical features |

## 5. Methodology and Implementation

### *5.1 Importing libraries*

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
```

Pandas and NumPy support data manipulation and numerical operations. Matplotlib and Seaborn are used for visualization, while StandardScaler is used for feature standardization.

### *5.2 Loading the dataset*

```
DATA_URL = "https://huggingface.co/datasets/DElmazi/Student_Performance/resolve/main/student-mat.csv"
df_original = pd.read_csv(DATA_URL, sep=";")
df = df_original.copy()

print("Shape:", df.shape)
df.head()
```

The CSV is read into a Pandas DataFrame. The separator is specified as a semicolon because the source file uses semicolons between fields. A copy is created so that the original loaded data remains unchanged during preparation. The expected dimensions are 395 rows and 33 columns.

### *5.3 Inspecting the dataset*

```
print(df.shape)
print(df.columns.tolist())
df.info()
display(df.head())
display(df.tail())
```

The shape, column names, first and last records, and data types are examined. The info() method also reports the number of non-null values in each column. This provides an initial understanding of the dataset structure and helps identify possible data-quality issues.

### *5.4 Descriptive statistical analysis*

```
display(df.describe())
```

The describe() method summarizes numerical columns using count, mean, standard deviation, minimum, quartiles, median, and maximum. These measures help understand central tendency, spread, and the range of numerical values.

### 5.5 Missing-value inspection

```
missing_values = df.isnull().sum()
print(missing_values)
print("Total missing values:", df.isnull().sum().sum())
```

Missing values are counted column by column. Depending on the data and the reason for missingness, common handling methods include mean or median imputation for numerical data, mode imputation for categorical data, or removal of unsuitable records. The UCI dataset documentation reports no missing values for this dataset; the notebook nevertheless checks the loaded file rather than assuming this result.

### 5.6 Duplicate-record inspection and removal

```
duplicate_count = df.duplicated().sum()
print("Duplicate rows:", duplicate_count)

df = df.drop_duplicates()
print("Shape after duplicate removal:", df.shape)
```

Duplicate rows are identified using duplicated() and removed using drop_duplicates(). Repeated records can give some observations unintended extra weight. If no duplicates are present, the dataset size remains unchanged.

### 5.7 Basic cleaning and consistency

```
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()
```

Leading and trailing spaces are removed from text columns. This prevents values such as 'Urban' and 'Urban ' from being treated as separate categories. Additional checks should confirm that categorical values belong to expected categories and that numerical fields have sensible ranges.

## 6. Exploratory Data Analysis (EDA)

EDA uses summary statistics and visualizations to understand distributions, identify unusual observations, and explore relationships between variables. The following plots are included in the notebook.

### 6.1 Histogram of final grades

```
plt.figure(figsize=(8, 5))
sns.histplot(df["G3"], bins=10, kde=True)
plt.title("Distribution of Final Student Grades")
plt.xlabel("Final Grade (G3)")
plt.ylabel("Number of Students")
plt.show()
```

The histogram shows how final mathematics grades are distributed across students. It helps identify commonly occurring grade ranges, the spread of grades, and whether the distribution is concentrated or skewed.

### 6.2 Scatter plot: study time versus final grade

```
plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x="studytime", y="G3")
plt.title("Study Time vs Final Grade")
plt.xlabel("Weekly Study Time")
plt.ylabel("Final Grade (G3)")
plt.show()
```

The scatter plot compares the weekly study-time category with the final grade. It can reveal patterns or variation in grades across study-time categories. Any visible association should not be interpreted as proof that study time alone causes a change in grades.

### 6.3 Box plot of final grades

```
plt.figure(figsize=(8, 5))
sns.boxplot(y=df["G3"])
plt.title("Box Plot of Final Grades")
plt.ylabel("Final Grade (G3)")
plt.show()
```

The box plot summarizes the median, quartiles, spread, and potential outliers in final grades. Potential outliers should be investigated in context rather than automatically deleted.

## 7. Categorical Encoding

Many data-analysis and machine-learning methods require numerical input. Categorical text features are therefore transformed using One-Hot Encoding.

```
categorical_columns = df.select_dtypes(
    include="object"
).columns.tolist()

df_encoded = pd.get_dummies(
    df,
    columns=categorical_columns,
    drop_first=True,
    dtype=int
)
```

One-Hot Encoding creates indicator columns for category values. Each indicator represents whether a record belongs to a particular category. The drop_first=True option omits one category from each encoded group, which can reduce redundant information in some modelling workflows. The encoded columns contain binary values.

## 8. Numerical Feature Scaling

Numerical features may have different ranges. Standardization places numerical input features on a comparable scale using the formula:

$$\mathbf{Z = (X - \mu) / \sigma}$$

Here, X is the original value, $\mu$ is the feature mean, $\sigma$ is the feature standard deviation, and Z is the standardized value. The transformed feature has approximately zero mean and unit standard deviation on the data used to fit the scaler.

```
numeric_columns = df_encoded.select_dtypes(
    include=np.number
).columns.tolist()

numeric_features = [
    col for col in numeric_columns if col != "G3"
]

scaler = StandardScaler()
df_encoded[numeric_features] = scaler.fit_transform(
    df_encoded[numeric_features]
)
```

The final grade G3 is left unchanged because it may be used as an outcome or target in later analysis. In a future predictive workflow, the scaler should be fitted only on the training data and then applied to validation or test data to avoid data leakage.

## 9. Saving the Prepared Dataset

```
output_file = "student_performance_prepared.csv"

df_encoded.to_csv(output_file, index=False)

print("Saved:", output_file)
print("Final shape:", df_encoded.shape)
```

The prepared DataFrame is exported as student_performance_prepared.csv. The index is excluded because it is not a dataset feature. The saved file can be reused for later analysis.

## 10. Summary of Data Preparation

| Stage | Action performed |
|---|---|
| Loading | Loaded the Mathematics CSV file into a DataFrame |
| Inspection | Checked dimensions, columns, records, and data types |
| Statistics | Generated descriptive statistics |
| Missing values | Checked each column for missing values |
| Duplicates | Detected and removed duplicate rows if present |
| Cleaning | Removed leading and trailing spaces from text fields |

| Stage | Action performed |
|---|---|
| EDA | Created histogram, scatter plot, and box plot |
| Encoding | Applied One-Hot Encoding to categorical text columns |
| Scaling | Applied StandardScaler to numerical input features, excluding G3 |
| Export | Saved the prepared data as a CSV file |

## 11. Observations

• The Mathematics dataset is expected to contain 395 student records and 33 original attributes.

• The data includes personal, family, educational, social, and academic information.

• Missing values and duplicate records are explicitly checked before transformation.

• Text-based categorical columns are converted into numerical indicator columns.

• Numerical input features are standardized, while G3 is retained in its original grade scale.

• The visualizations help examine final-grade distribution, study-time patterns, and potential outliers.

**Reporting note:** Insert the actual missing-value count, duplicate count, final encoded shape, and observations from the executed notebook. Do not assume numerical results that have not been produced by running the code.

## 12. Result

The Student Performance Mathematics dataset was selected and processed using Python. The workflow covered data inspection, descriptive statistics, missing-value and duplicate checks, basic text cleaning, exploratory visualizations, categorical encoding, and numerical feature scaling. The final prepared dataset is exported as student_performance_prepared.csv when the notebook is executed successfully.

## 13. Conclusion

This activity provided practical experience in preparing a raw tabular dataset for analysis. It demonstrated why data quality checks and transformations are important before drawing conclusions from data or using it in a machine-learning workflow. The histogram, scatter plot, and box plot supported initial exploration of student grades and study-time patterns. Categorical variables were transformed using One-Hot Encoding, and numerical input features were standardized using StandardScaler. Finally, the prepared dataset was saved in CSV format. Overall, the activity established a foundation in data preprocessing and exploratory data analysis.

## 14. References


• Cortez, P., & Silva, A. (2008). Using Data Mining to Predict Secondary School Student Performance. 5th Future Business Technology Conference.

• UCI Machine Learning Repository, Student Performance Dataset: https://archive.ics.uci.edu/dataset/320/student+performance

• Pandas documentation: https://pandas.pydata.org/docs/

• NumPy documentation: https://numpy.org/doc/

• Matplotlib documentation: https://matplotlib.org/stable/

• Seaborn documentation: https://seaborn.pydata.org/

• Scikit-learn documentation: https://scikit-learn.org/stable/